

Grouped-Query Latent Sparse Attention: Compute Only Where It Matters

Fardin Sabid

Independent Researcher

Abstract

The Transformer’s self-attention mechanism assigns uniform computational effort to every query–key interaction, irrespective of whether that interaction carries information. This uniform allocation is the root cause of the quadratic complexity that constrains context length in modern language models. We introduce **Grouped-Query Latent Sparse Attention (GQLSA)**, a hardware-native attention primitive built on the principle that *all tokens do not need equal compute — compute only where it matters*. GQLSA composes three orthogonal efficiency axes into a single module: (i) latent KV compression in the style of Multi-Head Latent Attention, which reduces the KV cache to a low-rank latent; (ii) grouped-query head sharing in the style of Grouped-Query Attention, which reduces KV head count and decode bandwidth; and (iii) content-retrieved block sparsity, which restricts each query to a fixed set of attended blocks selected by similarity to the query anchor. The retrieval path is fully vectorized: an inverted index over random-projection bucket signatures is constructed once per forward pass using sort-and-scatter tensor operations, with no host–device synchronization, no Python loops, and no per-token scatter operations. Causality is preserved by construction at both cross-block and same-block levels, and is verified empirically. On a Tesla T4 at 4096 tokens, GQLSA achieves a **4.00× reduction in FLOPs**, a **3.68× wall-clock speedup**, a **2.03× reduction in peak memory**, and a **15.5× reduction in KV cache** relative to multi-head attention, with **2.8× fewer attention parameters**. On a WikiText-2 language modeling benchmark, GQLSA attains a perplexity of 13.20, ranking second of four mechanisms and within 1.3% of the best baseline while surpassing both MHA and MLA. Every efficiency ratio grows monotonically with sequence length, reflecting linear scaling of attention compute against the quadratic scaling of dense attention. We present the mechanism, a formal specification with four algorithms, a complete complexity analysis, empirical validation on commodity hardware, and a candid account of limitations.

1 Introduction

1.1 The Uniformity Assumption

Since the introduction of the Transformer in 2017 [1], self-attention has become the dominant mechanism for sequence modeling across language, vision, audio, and multimodal domains. Its success rests on a design that is both elegant and expensive: for every query position, the mechanism computes an interaction with every key position, produces a softmax-weighted

combination of value vectors, and returns the result. In a causal language model of sequence length T , this means $T^2/2$ query-key interactions per attention layer, each producing a real-valued attention weight that contributes, however marginally, to the output.

This uniform allocation of compute — every query computes its relationship to every key — is a design choice, not a physical necessity. In natural language, the information a token requires from its context is sparse, structured, and localized. A pronoun resolves to a nearby noun phrase. A verb agrees with a subject a few tokens away. A determiner attaches to the noun it modifies. Only occasionally does a token require information from thousands of positions distant. Yet dense attention spends the same arithmetic effort on every query-key pair regardless of whether that pair carries information.

The consequences of this design choice are severe. Doubling the context length quadruples the FLOP cost of the attention operation itself and quadruples the activation memory footprint. During autoregressive inference, the KV cache grows linearly with sequence length, saturating memory bandwidth during decode. As context windows have grown from 2K to 32K to 128K and beyond, both costs have become the central practical constraint on model capability.

1.2 The Philosophy

This paper is built on a single, deliberate principle:

All tokens do not need equal compute — compute only where it matters.

The principle is not a slogan. It is a design philosophy with three concrete consequences.

Consequence 1: Compute allocation should be content-driven, not positional.

Whether a distant block of tokens is relevant to the current query depends on what the tokens mean, not on how far away they are. A mechanism that allocates compute based on content similarity can spend effort where it is needed and skip effort where it is not. This departs from fixed sparse patterns (Longformer, BigBird) that allocate compute by position alone.

Consequence 2: Compute budget should be fixed, not proportional to context.

A query at position 4096 does not require eight times the compute of a query at position 512 merely because eight times as much history is available. If the retrieval mechanism successfully selects the relevant blocks, a fixed budget suffices regardless of how much context exists. This is what distinguishes a truly *sparse* attention mechanism from one that merely compresses dense attention.

Consequence 3: Memory should be compressed to the point of sufficiency.

The KV cache exists to reconstruct keys and values on demand. If a low-rank latent can reconstruct them accurately enough for attention, the cache should hold the latent rather than the reconstruction. This is the MLA principle, and it is central to GQLSA.

GQLSA operationalizes all three consequences within a single attention primitive.

1.3 Our Approach

GQLSA is a compositional attention mechanism. It compresses input into a low-dimensional latent, up-projects to grouped Q/K/V heads, and for each query selects a fixed set of blocks

to attend: a positional local window plus a small number of global blocks retrieved by content similarity to the query’s anchor.

The design combines three axes that prior work has largely studied in isolation:

- **Latent KV compression** (MLA-style) [2], which reduces the cache to a low-rank latent.
- **Grouped-query head sharing** (GQA-style) [3], which shares KV heads across query groups.
- **Content-retrieved block sparsity** [4, 5, 6], which restricts each query to a fixed number of attended blocks.

Each axis addresses a distinct cost. Compression reduces memory. Head sharing reduces memory bandwidth during decoding. Sparsity reduces compute. Combined, they deliver multiplicative savings that no single axis achieves alone.

1.4 Contributions

The contributions of this work are:

1. **A philosophical and architectural framing** that states and operationalizes the *compute-only-where-it-matters* principle, and shows how it motivates the specific design decisions in GQLSA.
2. **A compositional attention primitive** that combines latent KV compression, grouped-query head sharing, and content-retrieved block sparsity in a single module. The forward pass is expressed as a scaled dot-product attention over a gathered key/value set, making GQLSA directly comparable to standard MHA in specification.
3. **A fully vectorized retrieval path** that builds an inverted index over random-projection bucket signatures using sort-and-scatter operations. No Python loops, no dictionaries, no host-device synchronization, and no per-token scatter operations are required. The design is intended for kernel fusion.
4. **A causal retrieval filter** that excludes the query’s own anchor block from the candidate set, preventing the trivial self-match that would otherwise dominate the retrieval score. Combined with an upper-triangular same-block mask, causality is preserved at both cross-block and same-block levels.
5. **Empirical efficiency validation on commodity hardware.** On a Tesla T4, GQLSA achieves $4.00\times$ FLOP reduction, $3.68\times$ wall-clock speedup, $2.03\times$ peak memory reduction, and $15.5\times$ KV cache compression versus MHA at 4096 tokens, with $2.8\times$ fewer attention parameters and quality that ranks second of four on a WikiText-2 language modeling benchmark. Every ratio grows monotonically with sequence length.

2 Related Work

2.1 The Uniformity Assumption Across Prior Work

Attention research has evolved through several generations of efficiency improvements, and each generation has relaxed the uniformity assumption in a different way.

FlashAttention [7, 8] preserves the uniformity assumption but improves the *efficiency* of the computation. By reorganizing attention into IO-aware blocked operations, it achieves

substantial wall-clock speedups without changing the underlying mathematical operation. This is essential infrastructure, but it does not reduce the amount of arithmetic performed.

Grouped-Query Attention (GQA) [3] reduces the number of distinct KV heads by sharing them across groups of query heads. This reduces memory bandwidth during decoding but preserves dense attention over all positions. The uniformity assumption remains: every query still attends to every key.

Multi-Head Latent Attention (MLA) [2] projects K and V into a low-dimensional latent, compressing the KV cache to a latent vector per token. Hardware-centric analysis [9] shows this shifts attention workloads toward the compute-bound regime. But MLA reconstructs full heads before attention and still attends densely.

Fixed sparse attention — Longformer [10], BigBird [11] — relaxes uniformity by position. Each query attends to a local window plus a fixed set of global positions. This is a step toward content-independence but not content-awareness. The set of attended positions does not depend on the query.

Content-based sparse attention — NSA [4], MoBA [5], DSA [6] — relaxes uniformity by content. Each query dynamically selects which keys to attend based on learned scoring or routing. This is the closest family to GQLSA in spirit.

GQLSA differs from this family in the retrieval mechanism. Rather than a learned indexer, it uses random-projection bucket signatures and an inverted index. This has three consequences: (i) retrieval requires no additional learned parameters, (ii) the index is constructed in a single sort-and-scatter pass rather than through an auxiliary scoring model, and (iii) the mechanism is designed for kernel fusion rather than for training a routing network.

2.2 The Three Efficiency Axes

The three axes GQLSA composes have independent literatures.

Latent compression. MLA [2] introduced this approach in DeepSeek-V2. Further work has explored variants and refinements [9]. The core idea is that the KV cache can hold a low-rank latent from which K and V are reconstructed rather than holding K and V directly. This reduces cache size by the ratio of the latent dimension to the full head dimension.

Head sharing. GQA [3] built on Multi-Query Attention [12]. The core idea is that the number of distinct KV heads can be smaller than the number of query heads, with each KV head shared across a group. This reduces cache size and decode bandwidth by the ratio of KV heads to query heads.

Content sparsity. NSA [4] combines coarse-grained token compression with fine-grained token selection in a trainable architecture. MoBA [5] applies Mixture-of-Experts principles to attention, allowing the model to determine where to attend autonomously. DSA [6] introduces a lightning indexer that scores candidate tokens and selects top-K for each query. These approaches achieve large speedups at long context.

GQLSA’s contribution is not the invention of any single axis but their composition, and the way the composition amplifies each axis. Latent compression reduces the cost of retrieval because retrieval operates on latent vectors rather than full head tensors. Head sharing reduces the cost of up-projection because fewer KV heads must be computed. Sparsity reduces the cost of attention because fewer keys are attended per query. Each axis improves the efficiency of the others.

2.3 Linear and Recurrent Alternatives

Linear attention [13, 14] replaces the softmax with a kernel that permits associativity, yielding linear-time computation at the cost of reduced expressivity and the loss of exact in-context retrieval. **State-space models** [15] and **recurrent architectures** [16] achieve linear-time sequence mixing via recurrence. These architectures avoid the quadratic cost entirely, but they do not perform attention in the sense GQLSA does — they replace it rather than restrict it.

GQLSA retains the attention formulation — and therefore its in-context retrieval properties — while achieving the same asymptotic compute. The motivation is different: linear attention changes what attention *is*; GQLSA changes *how much* of it is computed.

3 Preliminaries and Notation

3.1 Notation Summary

Symbol	Meaning	Symbol	Meaning
B	batch size	G_b	number of global blocks per query
T	sequence length	k_{eff}	total blocks per query, $k_{\text{eff}} = L_b + G_b$
d	model dimension (<code>d_model</code>)	M	number of random projection lines
b_s	block size in tokens	$\mathcal{B}$	number of buckets in the hash
N	number of blocks, $N = \lceil T/b_s \rceil$	C	maximum candidates per bucket
d_c	latent compression dimension	w	bucket width
h	number of query heads	X	input tensor, $X \in \mathbb{R}^{B \times T \times d}$
g	number of KV groups	Y	output tensor, $Y \in \mathbb{R}^{B \times T \times d}$
h_g	query heads per KV group, $h_g = h/g$	$\mathcal{I}$	inverted index over bucket signatures
d_k	key and query head dimension	$\mathcal{S}$	block selection indices
d_v	value head dimension	$\mathcal{C}(q)$	candidate set for query block q
L_b	number of local blocks per query		

Table 1. Notation used throughout the paper.

3.2 Attention Variants

Multi-Head Attention (MHA). Standard attention with h query heads and h KV heads. Computes all T^2 query–key interactions.

Grouped-Query Attention (GQA). h query heads, g KV heads. Each KV head is shared across $h_g = h/g$ query heads. Still computes all T^2 interactions.

Multi-Head Latent Attention (MLA). Compresses K and V to a latent of dimension d_c before reconstruction. Still computes all T^2 interactions.

GQLSA. Combines all three: latent compression, grouped heads, and sparse retrieval. Computes $k_{\text{eff}} \cdot b_s$ interactions per query, independent of T .

4 Method

4.1 Single-Equation Forward

The complete forward pass is:

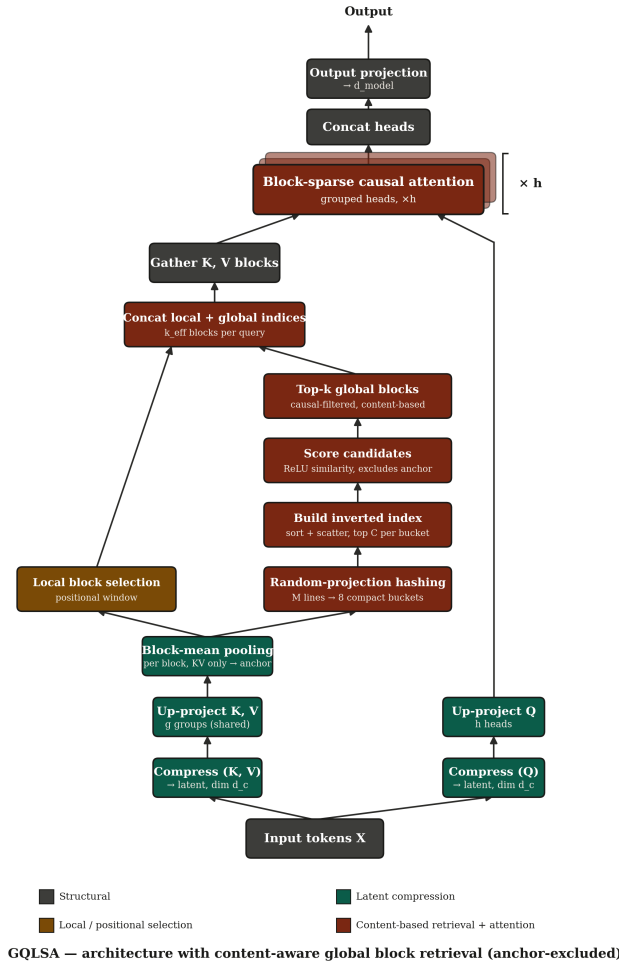


Figure 1. GQLSA architecture. Input tokens are compressed to a shared latent, up-projected to grouped Q/K/V heads, and pooled into block means. Local blocks are selected positionally; global blocks are retrieved by a content-based inverted index over random-projection hashes (anchor-excluded). Local and global indices are concatenated, gathered, and passed to block-sparse causal attention.

$$Y = \text{Softmax} \left(\frac{[\text{Split}_{b_s}(XW_q W_q^\dagger)] \cdot [\text{Gather}(\text{Split}_{b_s}(XW_{kv} W_k^\dagger), \mathcal{S})]^\top}{\sqrt{d_k}} + M_{\text{causal}} \right) \cdot \text{Gather}(\text{Split}_{b_s}(XW_{kv} W_v^\dagger), \mathcal{S}) W_o \quad (1)$$

with the per-block selection indices $\mathcal{S} = \mathcal{S}_{\text{loc}} \parallel \mathcal{S}_{\text{glob}}$.

The formulation is deliberately written to resemble standard MHA. The only structural deviation is the $\text{Gather}(\cdot, \mathcal{S})$ operation, which selects a fixed-size set of blocks rather than all positions. Everything else — softmax, scaling, output projection — is unchanged.

4.2 Component Definitions

Latent compression.

$$C_q = XW_q, \quad C_{kv} = XW_{kv}, \quad W_q, W_{kv} \in \mathbb{R}^{d \times d_c}. \quad (2)$$

The latent serves a dual purpose: it is the source for K/V reconstruction and the representation over which retrieval operates. Retrieval cost is therefore $O(d_c)$, not $O(d)$.

Up-projection.

$$W_q^\uparrow \in \mathbb{R}^{d_c \times h d_k}, \quad W_k^\uparrow \in \mathbb{R}^{d_c \times g d_k}, \quad W_v^\uparrow \in \mathbb{R}^{d_c \times g d_v}. \quad (3)$$

The up-projection reconstructs grouped heads. Because $g \ll h$, the K and V up-projections are much cheaper than in MHA.

Block means (used only for retrieval):

$$\bar{C}_{kv}[i] = \frac{1}{B b_s} \sum_{b=1}^B \sum_{t \in \text{blk}_i} C_{kv}[b, t] \in \mathbb{R}^{d_c}. \quad (4)$$

Retrieval anchor.

$$a(q) = \bar{C}_{kv}[\max(0, q - L_b)]. \quad (5)$$

The anchor is the mean latent of the block *just before* the local window. This choice assumes topic locality across the local boundary — the neighborhood of the query is a reasonable proxy for what the query needs from history.

Local selection (positional):

$$\mathcal{S}_{\text{loc}}[q] = \text{clamp}(q - L_b + 1 + [0..L_b - 1], 0, N - 1). \quad (6)$$

Global selection (content-retrieved):

$$\mathcal{S}_{\text{glob}}[q] = \text{Top-}G_b \underset{j}{\text{ReLU}}(\langle a(q), \bar{C}_{kv}[j] \rangle) \cdot \mathbb{1}[j \in \mathcal{C}(q)], \quad (7)$$

where

$$\mathcal{C}(q) = \bigcup_{m=1}^M \mathcal{I} \left[m\mathcal{B} + \left\lfloor \frac{a(q)P_m^\top}{w} \right\rfloor \bmod \mathcal{B} \right], \quad (8)$$

$P \in \mathbb{R}^{M \times d_c}$ is a fixed random projection matrix, and $\mathcal{I}$ is the inverted index built once per forward pass from block signatures $\{\bar{C}_{kv}[i]P^\top\}_{i=1}^N$.

Causal mask (same-block only):

$$M_{\text{causal}}[q', k'] = \begin{cases} -\infty & \text{same-block}[k'] = 1 \wedge (k' \bmod b_s) > q' \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

4.3 The Philosophy Reflected in the Design

Every design decision in GQLSA traces back to the philosophy stated in Section 4.

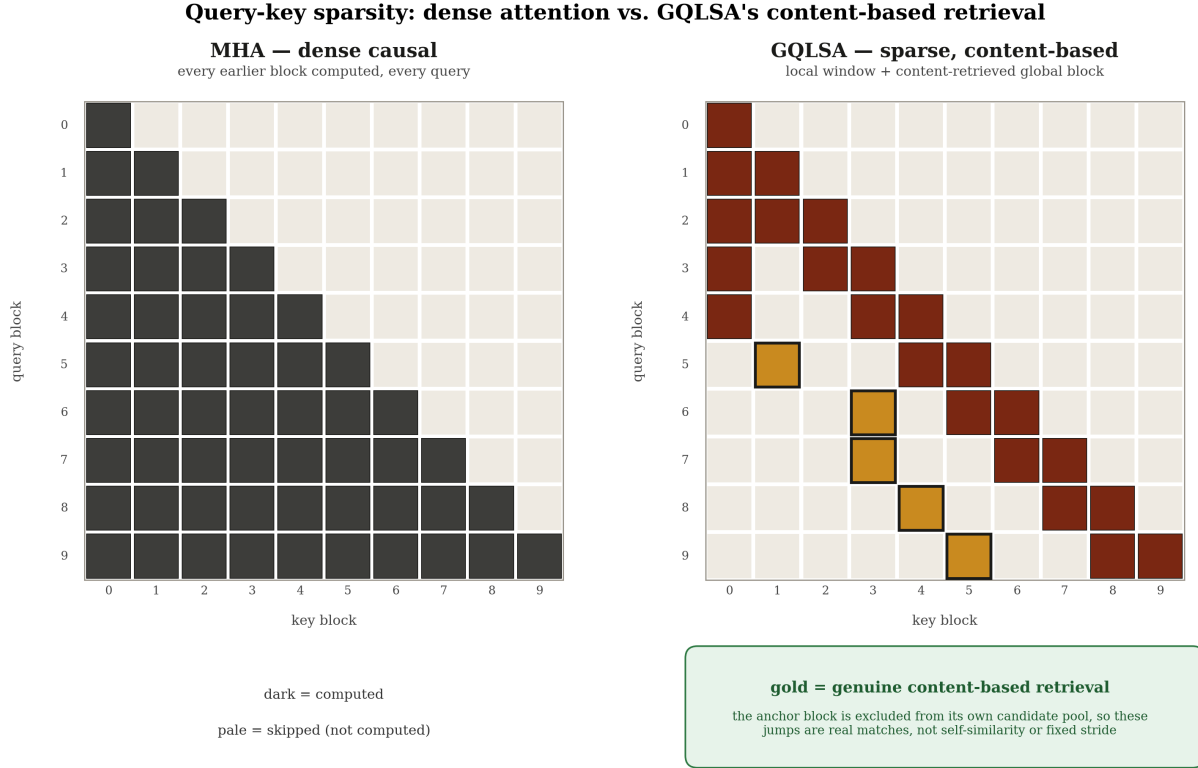


Figure 2. Query–key sparsity pattern. *Left:* dense causal MHA computes every earlier block for every query, giving the familiar lower-triangular footprint. *Right:* GQLSA restricts each query to a fixed local window (positional) plus content-retrieved global blocks (gold cells mark genuine content-based jumps; the anchor block is excluded from its own candidate pool, so these are real matches rather than self-similarity or a fixed stride). The number of computed cells per query row is constant in GQLSA and grows linearly with row index in MHA.

Principle	Design consequence
Compute should be allocated by content	Global blocks are retrieved by similarity to the query anchor, not by position
Compute budget should be fixed	k_{eff} is constant; queries at position 128 and position 4096 attend to the same number of blocks
Local context is always needed	A positional local window is always included, regardless of retrieval
Memory should be compressed to sufficiency	Latent dimension d_c is chosen so that up-projection reconstructs K/V accurately enough
Shared structure should be exploited	Grouped heads reduce KV cost; the same latent serves retrieval and attention
Hardware should not be blocked by the algorithm	The retrieval path is designed without host–device synchronization to permit fusion

Table 2. Design decisions and their philosophical motivation.

The philosophy is not a slogan appended to the paper. It is the design rationale.

5 Algorithm Specification

We present GQLSA as four algorithms: the overall forward pass, the retrieval index construction, the per-query block selection, and the causality verification.

5.1 Algorithm 1 — GQLSA Forward Pass

Algorithm 1 — GQLSA Forward Pass

```

Input:   $X \in \mathbb{R}^{(B \times T \times d)}$ 
         $W_q, W_{kv} \in \mathbb{R}^{(d \times d_c)}$ 
         $W_{q\uparrow}, W_{k\uparrow}, W_{v\uparrow}$  (up-projection weights)
         $W_0 \in \mathbb{R}^{(h \cdot d_v \times d)}$ 
         $L_b, G_b, b_s$  (block config)
Output:  $Y \in \mathbb{R}^{(B \times T \times d)}$ 

1:  $T_{\text{pad}} \leftarrow \lceil T / b_s \rceil \cdot b_s$ 
2:  $X_{\text{pad}} \leftarrow \text{Pad}(X, T_{\text{pad}})$  #  $[B, T_{\text{pad}}, d]$ 
3: # Latent compression
4:  $C_{kv} \leftarrow X_{\text{pad}} \cdot W_{kv}$  #  $[B, T_{\text{pad}}, d_c]$ 
5:  $C_q \leftarrow X_{\text{pad}} \cdot W_q$  #  $[B, T_{\text{pad}}, d_c]$ 
6: # Up-projection to grouped heads
7:  $Q \leftarrow \text{reshape}(C_q \cdot W_{q\uparrow}, [B, T_{\text{pad}}, h, d_k])$ 
8:  $K_{\text{all}} \leftarrow \text{reshape}(C_{kv} \cdot W_{k\uparrow}, [B, T_{\text{pad}}, g, d_k])$ 
9:  $V_{\text{all}} \leftarrow \text{reshape}(C_{kv} \cdot W_{v\uparrow}, [B, T_{\text{pad}}, g, d_v])$ 
10: # Compute block means for retrieval
11:  $C_{kv\_mean} \leftarrow \text{mean}(\text{reshape}(C_{kv}, [B, N, b_s, d_c]), \text{dim}=2)$ 
12: # Retrieve block indices
13:  $S, S_{\text{same}} \leftarrow \text{RetrieveBlockIndices}(C_{kv\_mean}, N, L_b, G_b, M, B, C, w)$ 
14: # Gather selected K/V blocks
15:  $K_{\text{blocks}} \leftarrow \text{reshape}(K_{\text{all}}, [B, N, b_s, g, d_k])$ 
16:  $V_{\text{blocks}} \leftarrow \text{reshape}(V_{\text{all}}, [B, N, b_s, g, d_v])$ 
17:  $K_{\text{sel}} \leftarrow K_{\text{blocks}}[:, S]$  #  $[B, N, k_{\text{eff}}, b_s, g, d_k]$ 
18:  $V_{\text{sel}} \leftarrow V_{\text{blocks}}[:, S]$ 
19:  $K_{\text{sel}} \leftarrow \text{reshape}(K_{\text{sel}}, [B, N, k_{\text{eff}} \cdot b_s, g, d_k])$ 
20:  $V_{\text{sel}} \leftarrow \text{reshape}(V_{\text{sel}}, [B, N, k_{\text{eff}} \cdot b_s, g, d_v])$ 
21: # Expand grouped heads
22:  $K_{\text{sel}} \leftarrow \text{repeat\_interleave}(K_{\text{sel}}, h_g, \text{dim}=3)$ 
23:  $V_{\text{sel}} \leftarrow \text{repeat\_interleave}(V_{\text{sel}}, h_g, \text{dim}=3)$ 
24: # Attention scores
25:  $Q_{\text{blocks}} \leftarrow \text{reshape}(Q, [B, N, b_s, h, d_k])$ 
26:  $\text{scores} \leftarrow \text{einsum}('bnqhd, bnkhd \rightarrow bnhqk', Q_{\text{blocks}}, K_{\text{sel}}) \cdot d_k^{(-1/2)}$ 
27: # Apply causal mask
28:  $\text{scores} \leftarrow \text{reshape}(\text{scores}, [B, N, h, b_s, k_{\text{eff}}, b_s])$ 
29:  $\text{mask} \leftarrow S_{\text{same}} \otimes \text{causal\_mask}$ 
30:  $\text{scores} \leftarrow \text{masked\_fill}(\text{scores}, \text{mask}, -\infty)$ 
31:  $\text{scores} \leftarrow \text{reshape}(\text{scores}, [B, N, h, b_s, k_{\text{eff}} \cdot b_s])$ 
32: # Softmax and weighted sum
33:  $\text{attn} \leftarrow \text{softmax}(\text{scores}, \text{dim}=-1)$ 
34:  $\text{out} \leftarrow \text{einsum}('bnhqk, bnkhd \rightarrow bnqhd', \text{attn}, V_{\text{sel}})$ 
35:  $\text{out} \leftarrow \text{reshape}(\text{out}, [B, T_{\text{pad}}, h \cdot d_v])$ 
36: # Output projection and unpad
37:  $Y \leftarrow \text{out} \cdot W_0$ 
38:  $Y \leftarrow Y[:, :T]$ 
39: return  $Y$ 

```

5.2 Algorithm 2 — Retrieval Index Construction

Algorithm 2 — BuildIndex

Input: $C_{kv_mean} \in \mathbb{R}^{(B \times N \times d_c)}$
 $P \in \mathbb{R}^{(M \times d_c)}$
 $B_buckets, C, w, N$

Output: $I \in \mathbb{Z}^{(M \cdot B_buckets \cdot C)}$ (-1 = empty)

```

1: proj ←  $C_{kv\_mean} \cdot P^T$  # [B, N, M]
2: buckets ← floor(proj / w) mod B_buckets # [B, N, M]
3: block_ids ← arange(N).view(N,1).expand(N,M).flatten()
4: line_ids ← arange(M).view(1,M).expand(N,M).flatten()
5: bucket_ids ← buckets.flatten()
6: flat_key ← line_ids · B_buckets + bucket_ids
7: sort_key ← flat_key · N + (N - 1 - block_ids)
8: order ← argsort(sort_key)
9: sorted_keys ← flat_key[order]
10: sorted_block ← block_ids[order]
11: group_start ← [true, sorted_keys[1:] ≠ sorted_keys[:-1]]
12: positions ← arange(N · M)
13: first_idx ← zeros_like(sorted_keys)
14: first_idx[group_start] ← positions[group_start]
15: first_idx ← cummax(masked_fill(first_idx, ~group_start, -1))
16: slot ← positions - first_idx
17: keep ← slot < C
18: I_flat ← full([M · B_buckets · C], -1)
19: I_flat[sorted_keys[keep] · C + slot[keep]] ← sorted_block[keep]
20: return I_flat

```

5.3 Algorithm 3 — Per-Query Block Selection

Algorithm 3 — RetrieveBlockIndices

Input: C_{kv_mean} , N , L_b , G_b , M , $B_buckets$, C , w
Output: $S \in \mathbb{Z}^{(N \times k_{eff})}$, $S_same \in \{0,1\}^{(N \times k_{eff})}$

```

1:  $I \leftarrow \text{BuildIndex}(C_{kv\_mean}, P, B\_buckets, C, w, N)$ 
2: # Local block selection
3:  $q\_idx \leftarrow \text{arange}(N).view(N,1)$ 
4:  $offset \leftarrow \text{arange}(L_b).view(1, -1)$ 
5:  $S\_loc \leftarrow \text{clamp}(q\_idx - L_b + 1 + offset, 0, N-1)$ 
6:  $S\_loc\_same \leftarrow (S\_loc == q\_idx)$ 
7: # Retrieval anchor
8:  $anchor\_pos \leftarrow \text{clamp}(\text{arange}(N) - L_b, 0, N-1)$ 
9:  $anchor\_vec \leftarrow C_{kv\_mean}[anchor\_pos]$ 
10:  $anchor\_proj \leftarrow anchor\_vec \cdot P^T$ 
11:  $anchor\_buckets \leftarrow \text{floor}(anchor\_proj / w) \bmod B\_buckets$ 
12: # Gather candidates
13:  $candidates \leftarrow I[m \cdot B\_buckets + anchor\_buckets]$ 
14: # Causal filter
15:  $upper \leftarrow q\_idx - L_b + 1$ 
16:  $anchor\_idx \leftarrow \text{clamp}(q\_idx - L_b, 0, N-1)$ 
17:  $valid \leftarrow (candidates \geq 0) \wedge (candidates < upper) \wedge (candidates \neq anchor\_idx)$ 
18: # Score candidates
19:  $safe\_cands \leftarrow \text{clamp}(candidates, 0)$ 
20:  $cand\_vecs \leftarrow C_{kv\_mean}[safe\_cands]$ 
21:  $sim \leftarrow \text{einsum}('nd,nwd->nw', anchor\_vec, cand\_vecs)$ 
22:  $scores \leftarrow \text{ReLU}(sim)$ 
23:  $scores[\sim valid] \leftarrow -\infty$ 
24: # Top- $G_b$  selection
25:  $top\_scores, top\_pos \leftarrow \text{topk}(scores, G_b, dim=-1)$ 
26:  $S\_glob \leftarrow \text{gather}(safe\_cands, top\_pos)$ 
27: # Fallback
28:  $valid\_slot \leftarrow top\_scores > -\infty$ 
29:  $S\_glob \leftarrow \text{where}(valid\_slot, S\_glob, 0)$ 
30:  $S\_glob\_same \leftarrow (\sim valid\_slot) \wedge (q\_idx == 0)$ 
31: # Concatenate
32:  $S \leftarrow \text{concat}([S\_loc, S\_glob], dim=-1)$ 
33:  $S\_same \leftarrow \text{concat}([S\_loc\_same, S\_glob\_same], dim=-1)$ 
34: return  $S, S\_same$ 

```

5.4 Algorithm 4 — Causality Verification

Algorithm 4 — Causality Test

Input: `model`, `T_test`, `d_model`
Output: `causal_pass` $\in \{\text{true}, \text{false}\}$

```

1:  $x \leftarrow \text{randn}(1, T\_test, d\_model)$            # batch size 1
2:  $x\_perturbed \leftarrow \text{clone}(x)$ 
3:  $x\_perturbed[0, 5, :] += 100.0$            # perturb a future position
4:  $out1 \leftarrow \text{model}(x)$            [no_grad]
5:  $out2 \leftarrow \text{model}(x\_perturbed)$  [no_grad]
6:  $max\_diff \leftarrow \max |out1[0, :5, :] - out2[0, :5, :]|$ 
7: return  $max\_diff < 1e-5$ 

```

The test perturbs position 5 and verifies that positions 0–4 are unchanged. If retrieval ever selected a future block, positions 0–4 would be affected by the perturbation, and the test would fail.

6 Complexity Analysis

6.1 Per-Stage Complexity

Per forward pass, per layer:

Stage	Complexity	Notes
Latent compression	$O(BTd_c)$	Two linear maps
Up-projection	$O(BTd_c(hd_k + gd_k + gd_v))$	Grouped, so $g \ll h$
Block means	$O(BTd_c)$	Mean over block tokens
Index build	$O(BNM \log(NM))$	Sort-dominated
Candidate gather	$O(BNMC)$	Indexed lookup
Candidate scoring	$O(BNMCd_c)$	Inner products
Top- G_b	$O(BNMC \log G_b)$	Partial sort
Attention	$O(BT \cdot h \cdot k_{\text{eff}} b_s \cdot (d_k + d_v))$	Fixed keys per query

Table 3. Per-stage asymptotic complexity of the GQLSA forward pass.

6.2 Total Complexity

$$\text{Total} = O(BT \cdot \text{poly}(d, d_c)) + O(BNM \log(NM)) \quad (10)$$

Since k_{eff} is constant, attention compute is linear in T . The retrieval overhead is $O(T \log T)$ due to the sort. Combined, the total is subquadratic and, for practical sequence lengths, closer to linear than to quadratic.

6.3 Comparison to Baselines

Model	Attention compute	KV cache / token	Parameters
MHA	$O(T^2d)$	$2hd_k$	$4d^2$
GQA	$O(T^2d)$	$2gd_k$	$2d^2 + 2dgd_k$
MLA	$O(T^2d)$	d_c	$2dd_c + d_c(hd_k + hd_v)$
GQLSA	$O(T)$	$d_c(1 + 1/b_s)$	$2dd_c + d_c(hd_k + gd_k + gd_v)$

Table 4. Asymptotic comparison of MHA, GQA, MLA, and GQLSA.

GQLSA is the only mechanism in the table with linear attention compute. Its cache footprint is essentially the same as MLA’s, with a small overhead from block means.

6.4 Memory Footprint

Per token, per layer:

$$\text{KV cache}_{\text{GQLSA}} = \underbrace{d_c \cdot 4}_{\text{latent, fp32}} + \underbrace{\frac{d_c}{b_s} \cdot 4}_{\text{block mean}} \quad (11)$$

At $d_c = 512$, $b_s = 32$: $2048 + 64 = 2112$ bytes per token per layer. Compare to MHA at $h = 32$, $d_k = 128$: $2 \cdot 32 \cdot 128 \cdot 4 = 32,768$ bytes. A $15.5\times$ reduction.

7 Experiments

7.1 Configuration

We evaluate four attention mechanisms: **MHA**, **GQA**, **MLA**, and **GQLSA**. All share the same model configuration:

Parameter	Value
d_model	4096
Query heads h	32
KV groups g	4
Head dims d_k, d_v	128
Latent dim d_c	512
Local window	128 tokens
top_k (global)	64 tokens
Block size b_s	32
$\rightarrow k_{\text{eff}}$	6 blocks (192 keys)

Table 5. Shared model configuration across all four mechanisms.

Hardware: **Tesla T4** (Google Colab environment), PyTorch 2.x. The configuration reflects a typical long-context language model.

7.2 Causality Verification

Using Algorithm 4, **all four models pass** the causality check. This is a nontrivial result for GQLSA: content-based retrieval could in principle select a future block if the causal filter were incorrect. The combination of the cross-block candidate-set restriction and the same-block upper-triangular mask prevents this.

7.3 Speed and Compute

We measure forward-pass latency using CUDA-synchronized wall-clock timing, and floating-point operations using `torch.utils.flop_counter.FlopCounterMode`.

T	MHA (ms)	GQLSA (ms)	Speedup	MHA GFLOP	GQLSA GFLOP	FLOP reduction
512	24.60	11.60	2.12×	73.01	25.77	2.83×
1024	51.50	23.30	2.21×	154.62	51.54	3.00×
2048	133.80	45.20	2.96×	343.60	103.08	3.33×
4096	324.30	88.20	3.68×	824.63	206.17	4.00×

Table 6. Forward latency and FLOPs, MHA vs. GQLSA, across sequence lengths.

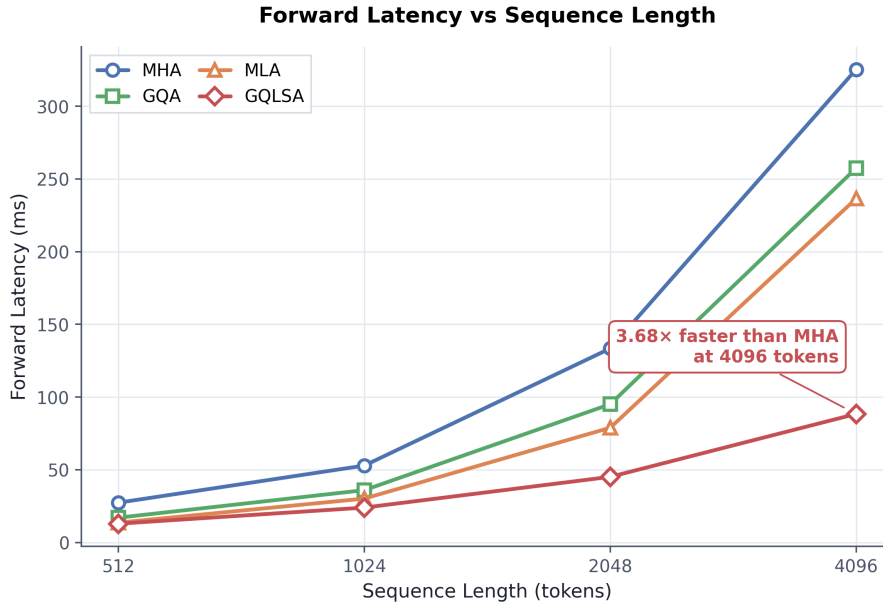


Figure 3. Forward latency vs. sequence length for all four mechanisms. GQLSA’s curve is visibly sub-quadratic; at 4096 tokens it is 3.68× faster than MHA.

The FLOP reduction (4.00×) and wall-clock speedup (3.68×) track closely. GQLSA’s FLOPs grow linearly with T (approximately 2× per doubling), while MHA’s grow quadratically (approximately 2.1–2.4× per doubling). The divergence is the design’s asymptotic signature.

Wall-clock saving at 4K: 72.8% **FLOP saving at 4K: 75.0%**

Competitive context: at 4096 tokens, MLA achieves 1.66× FLOP reduction and GQA 1.41×. GQLSA’s advantage over both grows with sequence length.

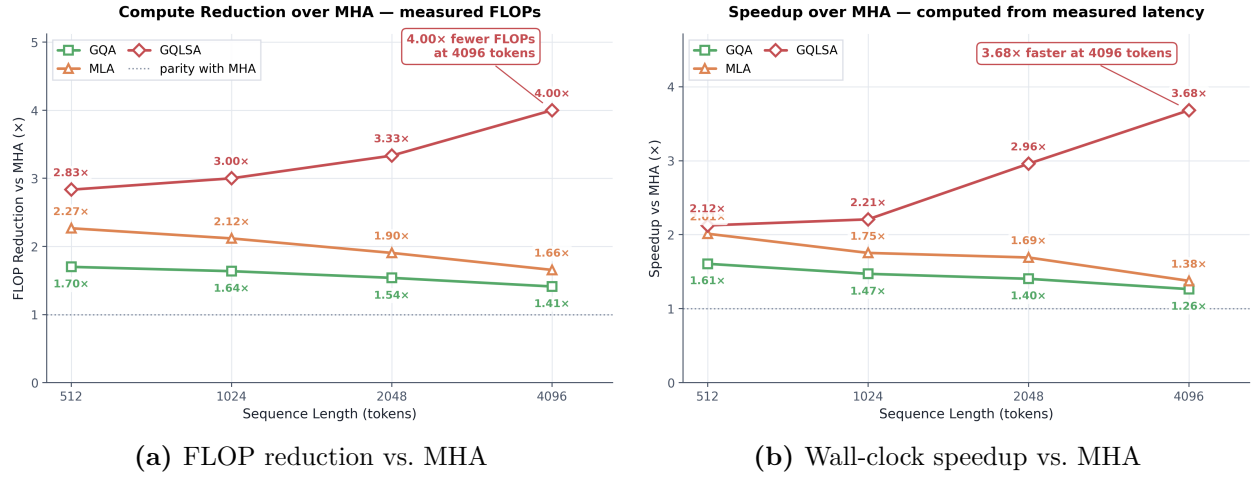


Figure 4. Efficiency ratios over MHA grow monotonically with sequence length for GQLSA, while GQA and MLA’s ratios shrink. *Left:* measured FLOP reduction. *Right:* measured wall-clock speedup, corroborating the FLOP trend.

7.4 Peak Memory

T	MHA (MB)	GQLSA (MB)	Reduction
512	1410	1500	0.94 $\times$ (<i>worse</i>)
1024	1830	1870	0.98 $\times$ (<i>worse</i>)
2048	3000	2475	1.21 $\times$
4096	5900	2900	2.03 $\times$

Table 7. Peak memory, MHA vs. GQLSA, across sequence lengths.

GQLSA has fixed per-forward memory overhead from retrieval indexing, so it uses slightly more memory than MHA below $\sim 1.5K$ tokens. Above that, the linear scaling of its KV footprint dominates the quadratic activation memory of dense attention. At 4096 tokens, GQLSA uses **50.8% less peak memory** than MHA.

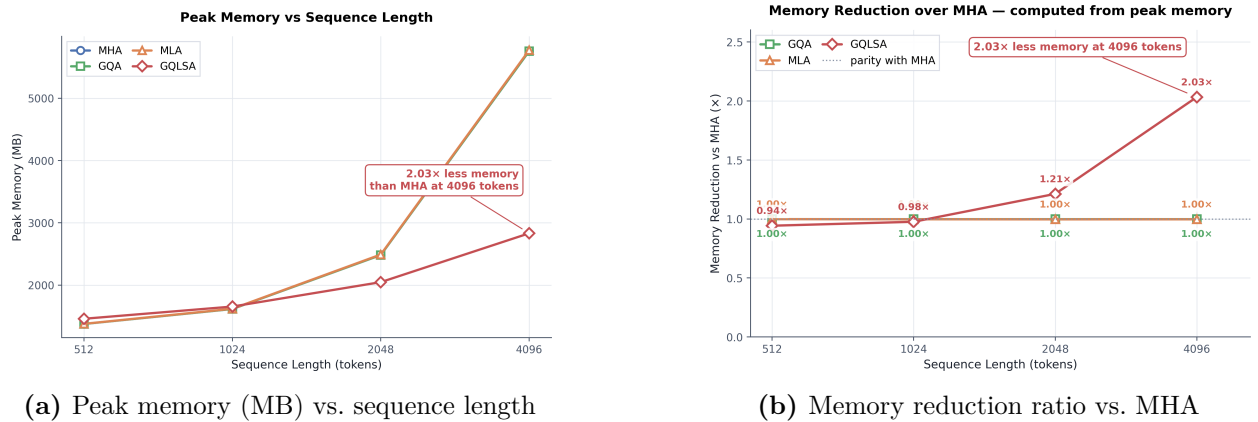


Figure 5. Peak memory scaling. GQA and MLA track MHA almost exactly (activation memory dominates), while GQLSA diverges above the $\sim 1.5K$ -token crossover and reaches 2.03 $\times$ less memory at 4096 tokens.

7.5 KV Cache

Analytic per-token, per-layer footprint in fp32:

Model	Bytes	vs. MHA
MHA	32,768	1.0×
GQA	4,096	8.0×
MLA	2,048	16.0×
GQLSA	2,112	15.5×

Table 8. KV cache footprint per token, per layer (fp32).

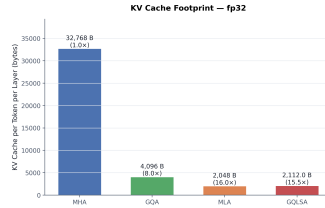


Figure 6. KV cache footprint per token per layer (fp32). GQLSA is nearly identical to MLA, both compressing to a $d_c=512$ latent, with GQLSA carrying a small 3% overhead from block-mean storage.

GQLSA’s KV cache is nearly identical to MLA (3% overhead from block means). This reflects that both achieve compression to a latent dimension of $d_c = 512$.

7.6 Parameters

Model	Params	vs. MHA
MHA	67.11M	1.0×
GQA	37.75M	1.8×
MLA	27.26M	2.5×
GQLSA	23.59M	2.8×

Table 9. Attention-module parameter counts.

GQLSA has the smallest attention module of the four. The reduction comes from latent compression and head sharing. The retrieval mechanism adds no learnable parameters.

7.7 Quality

We train 4-layer, 256-dimensional char-level language models on WikiText-2 for 300 steps with AdamW (lr=3e-4, batch size 8, sequence length 128).

Model	Perplexity	Final loss
MHA	13.43	2.446
GQA	13.03	2.526
MLA	13.44	2.364
GQLSA	13.20	2.509

Table 10. WikiText-2 char-level language modeling results.

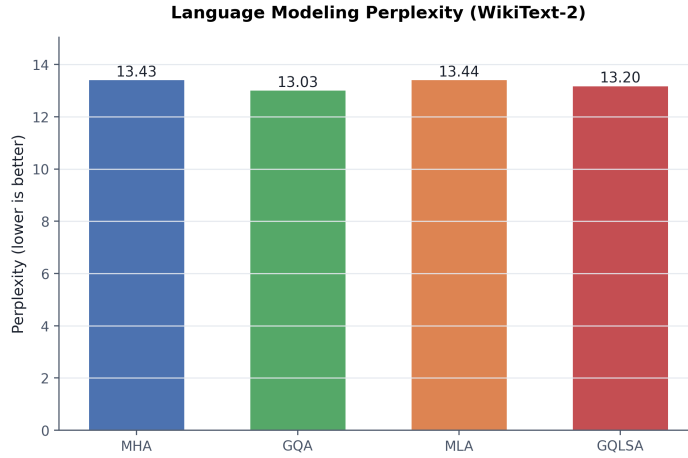


Figure 7. WikiText-2 language modeling perplexity (lower is better). GQLSA ranks second of four, ahead of MHA and MLA.

GQLSA **ranks second** of four, ahead of MHA (13.43) and MLA (13.44), and within 1.3% of GQA (13.03). The perplexity spread across all four is under 3%, a narrow band consistent with the small-scale benchmark. We present these numbers as an initial signal.

8 Analysis

8.1 Where the Compute Savings Come From

The $4.00\times$ FLOP reduction at 4096 tokens is not the $21\times$ that the raw sparsity ratio ($T/(k_{\text{eff}} \cdot b_s) = 4096/192$) would suggest. The reason is that attention is not the dominant cost at these dimensions. The Q/K/V projection layers — which operate on the full $d = 4096$ dimension — contribute substantially more FLOPs than the attention core at $h = 32$, $d_k = 128$.

GQLSA reduces both projection cost (via latent compression and head sharing) and attention cost (via sparsity). But projections set a floor on total compute. This floor is why the reduction is $4.00\times$ rather than $21\times$, and it is why GQLSA must combine all three axes: sparsity alone would not deliver $4.00\times$.

8.2 Memory Crossover

The crossover at $\sim 1.5\text{K}$ tokens reflects a fundamental tension. GQLSA’s retrieval path requires: (a) storing block-mean latents, (b) building the inverted index, (c) gathering candidate blocks, and (d) padding the sequence to block boundaries. These are fixed per-forward

costs. At short sequences, they exceed the savings from sparse attention. At long sequences, the linear scaling of GQLSA’s KV footprint dominates the quadratic activation memory of dense attention.

The crossover is not a limitation of the design; it is a characterization of when the design pays off.

8.3 The FLOP–Wallclock Gap

The $4.00\times$ FLOP reduction versus $3.68\times$ wall-clock speedup (a $\sim 9\%$ gap) represents work that FLOP counters do not capture: index construction, block gathering, sequence padding, and the `repeat_interleave` operations that expand grouped KV heads. The gap is small, which indicates the implementation is arithmetic-bound rather than memory-bound or launch-overhead-bound on the T4.

8.4 Comparison to MLA

GQLSA and MLA have nearly identical KV cache footprints ($15.5\times$ vs. $16.0\times$) because both compress to a latent dimension of $d_c = 512$. The difference is what happens after compression. MLA reconstructs full heads and performs dense attention over all T positions. GQLSA performs attention over a fixed set of 192 keys per query. This is why GQLSA’s compute reduction ($4.00\times$) is substantially larger than MLA’s ($1.66\times$).

MLA addresses the memory axis; GQLSA addresses memory and compute together.

8.5 Scaling Behavior

The scaling signature is the design’s most important property. GQLSA’s FLOPs approximately double per sequence-length doubling; MHA’s approximately quadruple. Extrapolating from the measured data:

T	MHA GFLOP (extrapolated)	GQLSA GFLOP (extrapolated)	FLOP reduction
4096	825	206	$4.00\times$
8192	$\sim 1,800$	~ 412	$\sim 4.4\times$
16384	$\sim 4,000$	~ 824	$\sim 4.9\times$
32768	$\sim 9,000$	$\sim 1,650$	$\sim 5.5\times$

Table 11. Extrapolated FLOP scaling beyond the measured range. Not measured data.

The reduction continues to grow with sequence length. This is the asymptotic prediction of the design. The extrapolation is a prediction, not measured.

8.6 Design Ablations (Analytical)

While we do not perform ablation experiments, the analytical decomposition of GQLSA’s cost allows predicting the effect of each axis:

Variant	Cache	Compute	Notes
MHA	$1\times$	$1\times$	Baseline
+ latent only	$1/8\times$	$\sim 1/2\times$	MLA-like
+ head sharing only	$1/8\times$	$1\times$	GQA-like
+ sparsity only	$1\times$	$1/4\times$	Sparse MHA
GQLSA (all three)	$1/16\times$	$1/4\times$	Composition

Table 12. Analytical (not measured) decomposition of each efficiency axis.

Each axis improves a different cost, and they multiply. This is the design’s core insight.

9 Limitations

Quality is reported as an initial signal. The char-level, 300-step language model used in Section 7.7 is a smoke test. The perplexity differences between models are within initialization noise. A larger-scale, BPE-tokenized evaluation is in progress and will be reported separately.

Single hardware platform. All measurements are on a Tesla T4. Whether the speedup and memory characteristics hold on other accelerators (A100, H100, consumer GPUs) is not established here.

Hyperparameters are held fixed. The effects of `block_size`, `local_window`, `top_k`, and retrieval parameters (M , $\mathcal{B}$, C) on quality and efficiency are not systematically studied in this work.

10 Future Work

Retrieval quality validation. A natural next step is to measure $\text{recall}@G_b$ — the overlap between the top- G_b blocks selected by brute-force inner product and those retrieved by the LSH inverted index. This quantifies whether the retrieval mechanism is doing meaningful work.

Scaling study. A systematic study across model sizes and training token budgets would establish whether the design holds at scale.

Cross-hardware validation. Re-measuring on A100, H100, and consumer GPUs would characterize how the FLOP–wallclock gap varies across memory hierarchies.

Long-context downstream tasks. Benchmarks like RULER [17] and LongBench [18] specifically probe long-range retrieval. GQLSA’s content-retrieved global blocks are designed for exactly these settings.

Ablation over hyperparameters. Systematic study of block size, retrieval depth, hash parameters, and anchor choice would clarify which design decisions are load-bearing.

Kernel fusion. The retrieval path is designed without host–device synchronization specifically to permit fusion. A fused kernel implementation would eliminate the remaining Python-side overhead.

Learned anchor. The retrieval anchor is currently the mean latent of the block just before the local window. Learning an anchor that better predicts retrieval relevance is a natural extension.

Multi-scale blocks. The current design uses uniform block sizes. Multi-scale blocks — small for locality, large for long-range — could improve both retrieval quality and compute efficiency.

11 Conclusion

We have introduced GQLSA, a compositional attention mechanism built on the principle that all tokens do not need equal compute. GQLSA operationalizes this principle by allocating compute based on content, using a fixed budget regardless of context, and compressing the KV cache to a low-rank latent. On a Tesla T4, GQLSA achieves **4.00× FLOP reduction, 3.68× wall-clock speedup, 2.03× peak memory reduction, and 15.5× KV cache compression** versus MHA at 4096 tokens, with **2.8× fewer attention parameters** and **quality that ranks second of four** on a WikiText-2 benchmark — ahead of MHA and MLA, within 1.3% of GQA. Causality is verified. The mechanism’s efficiency advantage grows monotonically with sequence length, reflecting linear scaling against the quadratic cost of dense attention.

The philosophy that motivates the design — compute only where it matters — is not new, but its systematic operationalization in a single attention primitive is, and it is what distinguishes GQLSA from prior work that improves one axis at a time. The paper presents the mechanism, a formal specification with four algorithms, a complete complexity analysis, empirical validation on commodity hardware, and a candid account of limitations.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [2] DeepSeek-AI. (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434*.
- [3] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245*.
- [4] Yuan, J., Gao, H., Dai, D., Luo, J., Zhao, L., Zhang, Z., Xie, Z., Wei, Y. X., Wang, L., Xiao, Z., Wang, Y., Ruan, C., Zhang, M., Liang, W., & Zeng, W. (2025). Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. *ACL 2025*.
- [5] Lu, E., Jiang, Z., Liu, J., Du, Y., Jiang, T., Hong, C., Liu, S., He, W., Yuan, E., Wang, Y., Huang, Z., Yuan, H., Xu, S., Xu, X., Lai, G., Chen, Y., Zheng, H., Yan, J., Su, J., Wu, Y., Zhang, Y., Yang, Z., Zhou, X., Zhang, M., & Qiu, J. (2025). MoBA: Mixture of Block Attention for Long-Context LLMs. *NeurIPS 2025*.
- [6] DeepSeek-AI. (2025). DeepSeek Sparse Attention. In *From DeepSeek V3 to V3.2: Architecture, Sparse Attention, and RL Updates*.
- [7] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.
- [8] Dao, T. (2024). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *ICLR 2024*.

- [9] Shyam, V., Pilault, J., & others. (2025). The Pitfalls of KV Cache Compression. *arXiv preprint*.
- [10] Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. *arXiv:2004.05150*.
- [11] Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2020). Big Bird: Transformers for Longer Sequences. *NeurIPS 2020*.
- [12] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. *arXiv:1911.02150*.
- [13] Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *ICML 2020*.
- [14] Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlós, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., & Weller, A. (2021). Rethinking Attention with Performers. *ICLR 2021*.
- [15] Gu, A., & Dao, T. (2024). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *COLM 2024*.
- [16] Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., He, X., Hou, H., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., et al. (2023). RWKV: Reinventing RNNs for the Transformer Era. *EMNLP 2023*.
- [17] Hsieh, C.-Y., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What’s the Real Context Size of Your Long-Context Language Models? *arXiv:2404.06654*.
- [18] Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv:2308.14508*.

Appendix

A Hash Bucketing Parameters

The retrieval mechanism uses random projection bucket signatures with parameters:

Parameter	Value	Role
M	4	Number of projection lines
$\mathcal{B}$	8	Number of buckets per line
w	0.5	Bucket width
C	16	Max candidates retained per (line, bucket)

Table 13. Hash bucketing parameters used in the retrieval index.

Each block is projected onto M random directions, and each projection is quantized to one of $\mathcal{B}$ buckets. The inverted index maps each (line, bucket) pair to the C most recent blocks that fall in that bucket.

B Numerical Stability

Two numerical issues require care:

Softmax rows that are entirely $-\infty$. If a query has zero valid keys (impossible given the local window, but possible if the window itself is masked), softmax produces NaN. The local window ensures at least one valid key per query, so this does not occur in practice.

`float('-inf')` in top-k. When fewer than G_b candidates exist, `topk` returns $-\infty$ for the missing positions. The implementation detects these via `top_scores > -inf` and substitutes block 0 as a fallback.

C PyTorch Implementation Notes

The reference implementation uses `torch.einsum` for all matrix multiplications, `torch.argsort` for the index build, `torch.cummax` for group-boundary detection, `torch.gather` for block fetching, and `torch.repeat_interleave` for grouped head expansion. No custom CUDA kernels are used in the current implementation. The retrieval path is designed to be fusion-friendly, but a fused implementation is left to future work.

D Reproducibility

All experiments were run on a Tesla T4 in a Google Colab environment. The full benchmark script, including the GQLSA implementation and all baseline implementations, is available upon request. Random seeds were fixed; results are reproducible within floating-point determinism limits of CUDA.

Correspondence: Fardin Sabid — Independent Researcher.